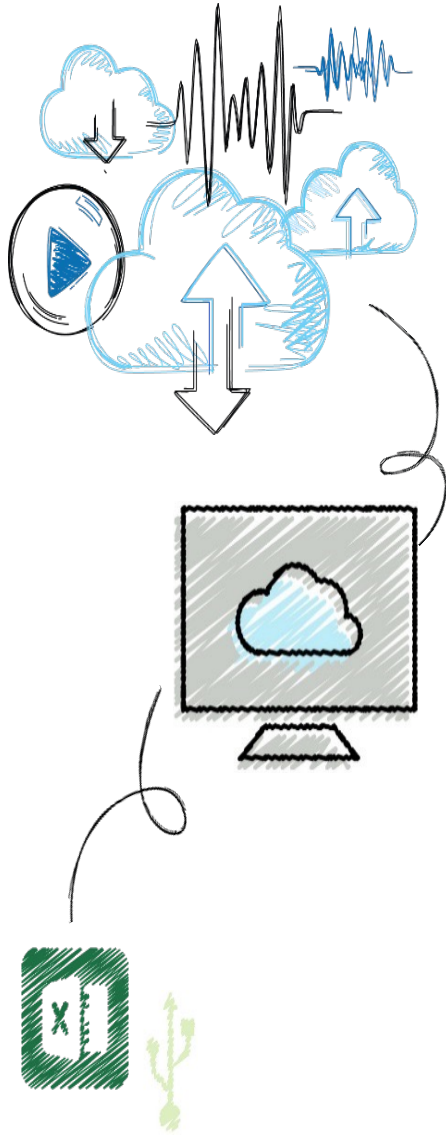


ICT703 Programming

Module 7

Design with Functions





Objectives

1. Explain why functions are useful in structuring code in a program
2. Employ top-down design to assign tasks to functions



What Functions Are and How They Work

- A function packages an algorithm in a chunk of code that you can call by name
- A function can be called from anywhere in a program's code, including code within other functions
- A function can receive data from its caller via arguments
- When a function is called, any expression supplied as arguments are first evaluated
- A function may have one or more **return** statements
- A function can return 1 or more values (via a tuple)



Programming Concepts

Four control flow

- Sequence
- Iteration
- Selection
- **Abstraction** – functions/methods/classes

Data



Functions as Abstraction Mechanisms

- An **abstraction** hides detail
 - Allows a person to view many things as just one thing
- We use abstractions to refer to the most common tasks in everyday life
 - For example, the expression “**doing my laundry**”
- Effective designers must invent useful abstractions to control complexity



The Syntax of Simple Function Definitions

- Definition of a function consists of header and body

```
def square(x):
```

```
    """Returns the square of x"""
```

```
    return x * x
```

- Syntax of a function definition:

```
def <function name>(<parameter-1>, ..., <parameter-n>):
```

```
<body>
```



Parameters and Arguments

- A parameter is the name used in the function definition for an argument that is passed to the function when it is called
- For now, the number and positions of arguments of a function call should match the number and positions of the parameters in the definition
- Some functions expect no arguments
 - They are defined with no parameters



Functions Eliminate Redundancy

- Functions serve as abstraction mechanisms by eliminating redundant, or repetitious, code

```
def summation(lower, upper):
```

```
    """Arguments: A lower bound and an upper bound
```

```
    Returns: the sum of the numbers from lower through upper """
```

```
    result = 0
```

```
    while lower <= upper:
```

```
        result += lower
```

```
        lower += 1
```

```
    return result
```

```
summation(1,4)
```

```
10
```

```
summation(50,100)
```

```
3825
```

```
# The summation of the numbers 1..4
```

```
# The summation of the numbers 50..100
```



Functions Hide Complexity

- Functions serve as abstraction mechanisms is by hiding complicated details
- For example, consider the previous **summation** function
 - The idea of summing a range of numbers is simple; the code for computing a summation is not
- A function call expresses the idea of a process to the programmer
 - Without forcing him/her to wade through the complex code that realizes that idea



Functions Support General Methods with Systematic Variations

- An algorithm is a **general method** for solving a class of problems
- The individual problems that make up a class of problems are known as **problem instances**
 - What are the problem instances of our summation algorithm?
- Algorithms should be general enough to provide a solution to many problem instances
 - A function should provide a general method with systematic variations



Functions Support the Division of Labor

- In a well-organized system, each part does its own job in collaborating to achieve a common goal
- In a computer program, functions can enforce a division of labor
 - Each function should perform a single coherent task
 - Example: Computing a summation
- Each of the tasks required by a system can be assigned to a function
 - Including the tasks of managing or coordinating the use of other functions



Problem Solving with Top-Down Design

- **Top-down design** starts with a global view of the entire problem and breaks the problem into smaller, more manageable subproblems
 - Process known as **problem decomposition**
- As each subproblem is isolated, its solution is assigned to a function
- As functions are developed to solve subproblems, solution to overall problem is gradually filled out
 - Process is also called **stepwise refinement**



The Return Statement

- Place a **return** statement at each exit point of a function when function should explicitly return a value
- Syntax:
return <expression>
- If a function contains no **return** statement, Python transfers control to the caller after the last statement in the function's body is executed
 - The special value **None** is automatically returned



Boolean Functions

- A **Boolean function** usually tests its argument for the presence or absence of some property
 - Returns **True** if property is present; **False** otherwise
- Example:

def odd(x):

"""Returns True if x is odd or False otherwise."""

if x % 2 == 1:

return True

else:

return False



Lifetime

- Variable's **lifetime**: Period of time when variable has memory storage associated with it
 - When a variable comes into existence, storage is allocated for it; when it goes out of existence, storage is reclaimed by the PVM
- Module variables come into existence when introduced and generally exist for lifetime of program that introduces or imports them
- Parameters and temporary variables come into existence when bound to values during call, but go out of existence when call terminates



Scope of Variables

- The formal parameter variables of a function are local variables
 - they exist only while the function is executing
 - The scope of a variable is the region of the program where it is usable
 - from where you create the variable to the end of that block of code
-

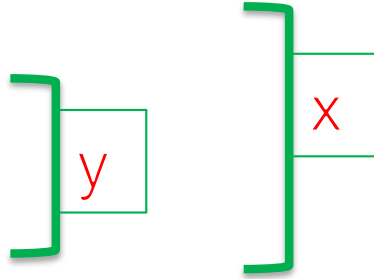


Scope of Variables

```
def square(x):
```

```
    y = x * x
```

```
    return y
```



```
toSquare = 10
```

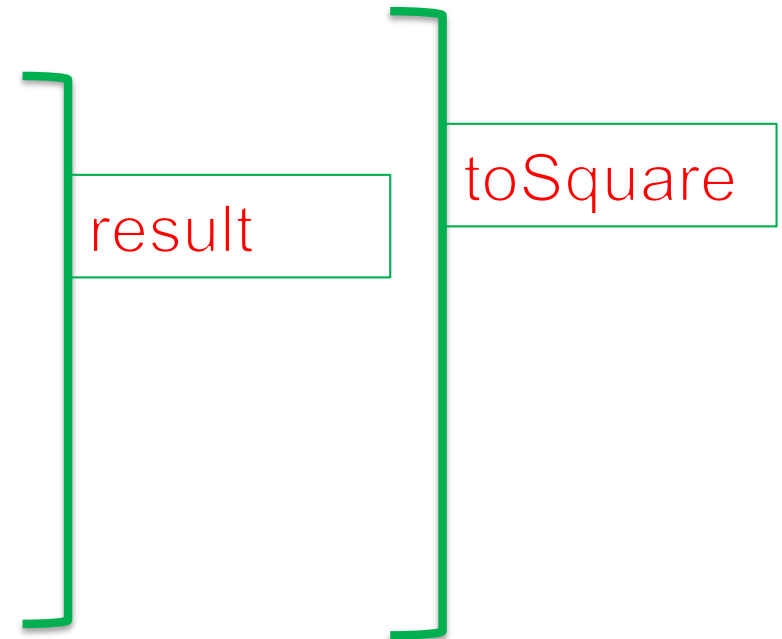
```
result = square(toSquare)
```

```
print(toSquare, " squared is ", result)
```

```
toSquare = 12
```

```
result = square(toSquare)
```

```
print(toSquare, " squared is ", result)
```





Functions and scopes

- Ideally, functions would only use their input variables, and local variables
 - So the function is reusable in other programs
 - easier to understand (no hidden values & variables!)
- But functions can access global variables

```
xGlobal = 10
```

```
def smallFunction():  
    print(xGlobal)
```



Scope rules for code inside a function

- You can read & write local variables
- You can read & write formal parameters
- You can read global variables
- You should NOT write to global variables



Some examples

```
def greet(name):
```

```
    """ This function greets to the person passed in as a parameter """
```

```
    print("Hello, " + name + ". Good morning!")
```

```
greet('Paul')
```

Hello, Paul. Good morning!



Some examples

```
def absolute_value(num):
```

```
    """This function returns the absolute value of the entered number"""
```

```
    if num >= 0:
```

```
        return num
```

```
    else:
```

```
        return -num
```

```
print(absolute_value(2))
```

```
print(absolute_value(-4))
```

2

4



Some examples

```
def introduction(message):
    '''This function prints the message'''
    print(message)

def get_Age():
    '''This function asks and return the person's age'''
    return(int(input("What is your age: ")))

def get_number(x):
    '''This function gets a random number (sort of)'''
    if x > 0:
        return (3)
    else:
        return (4)

introduction("Good morning")
yourAge = get_Age()
youLookLike = yourAge - get_number(10)
print("You only look ", youLookLike)
```



Some examples

```
def open_file(fileName):  
    '''This function opens the file name and return the file handler f'''  
    return f
```

```
def process_file(f):  
    '''This function loops through all the file handler f and does something. Returns True if is worked'''  
    return True
```

```
def close_file(f):  
    '''This function closes the file'''
```

```
f = open_file("temp.xls")  
process_file (f)  
close_file(f)
```
